

Product Requirements Document (PRD)

Product Name: CloudOptima (FinOps Enterprise) **Version:** 1.0 **Date:** June 2026

1. Product Overview

CloudOptima is a B2B SaaS dashboard designed for Cloud Architects, DevOps Engineers, and FinOps teams. It securely connects to a user's AWS account using least-privilege (Read-Only) IAM credentials to analyze cloud infrastructure, identify wasted spend, and provide actionable remediation steps.

2. Target Audience

- **Cloud Architects:** Looking for high-level overviews of infrastructure health and green computing impact.
- **FinOps Teams:** Needing to track estimated monthly bills, potential savings, and cost breakdowns by service.
- **DevOps Engineers:** Requiring specific resource IDs and reasons for idle/orphaned status to perform cleanup.

3. Core Features & Requirements

3.1 Authentication & Security (Firebase)

- **Secure Login/Signup:** Users must authenticate using email/password via Firebase Authentication.
- **Multi-Tenant Isolation:** AWS credentials must be strictly scoped to the individual user's session and securely wiped upon logout.
- **Encryption:** AWS Access Keys and Secret Keys must never be stored in plain text.

3.2 AWS Configuration (Settings Page)

- **Input Fields:** Users must provide an AWS Access Key ID and AWS Secret Access Key .

- **Live Validation:** The backend must validate the keys using the `@aws-sdk/client-sts ( GetCallerIdentityCommand )` before saving them.
- **Read-Only Enforcement:** The platform will visually instruct users to only provide keys with Read-Only policies attached.

3.3 Dashboard & Analytics

- **Top Metric Cards:**
 - *Estimated Monthly Bill:* Projected AWS running costs.
 - *Potential Monthly Savings:* Total cost of flagged waste (reducible waste %).
 - *Green Computing Impact:* Carbon offset metrics (kg CO2e/mo, equivalent trees planted, driving miles avoided).
- **Cost Breakdown by Service:** A visual Donut Chart displaying the percentage breakdown of active services (e.g., EC2, RDS, S3, EBS, Others).

3.4 Actionable Heuristics Engine

- **Resource Scanning:** The backend utilizes the `@aws-sdk/client-ec2` to fetch active and stopped infrastructure.
- **Waste Identification Table:** Displays a table of flagged resources including:
 - Resource Name / ID (e.g., `web-prod-01` , `i-0123456789abcdef0`)
 - Service & Status (e.g., EC2 - IDLE, EBS - ORPHANED)
 - Idle Reason (e.g., "Average CPU utilization is 2.1% for the last 14 days")
 - Potential Savings (\$ amount)
 - Action Dropdown (Remediation steps)

4. Technical Stack

- **Frontend:** React.js, Tailwind CSS (Dark Mode Enterprise UI layout).
- **Backend:** Node.js, Express.js.

- **Cloud Provider SDK:** AWS SDK for JavaScript v3.
- **Authentication:** Firebase Auth.

5. User Flow

1. User creates an account and logs in.
2. User is directed to the Settings page to input AWS IAM Read-Only keys.
3. System validates keys via Node.js backend.
4. On success, keys are temporarily secured in scoped storage.
5. User navigates to Dashboard.
6. Frontend triggers a fetch request to the Node.js backend.
7. Backend queries AWS, processes the heuristics, and returns the data payload.
8. Dashboard hydrates with live cost and resource data.